22AIE111 OBJECT ORIENTED PROGRAMMING IN JAVA

S2 B.Tech CSEAI

Department of Computer Science and Engineering

Amrita School of Computing, Amritapuri Campus

CASE STUDY

PHASE – 2 (Design)

Architectural Modelling (CO2)

Group No : 5

Team Members :

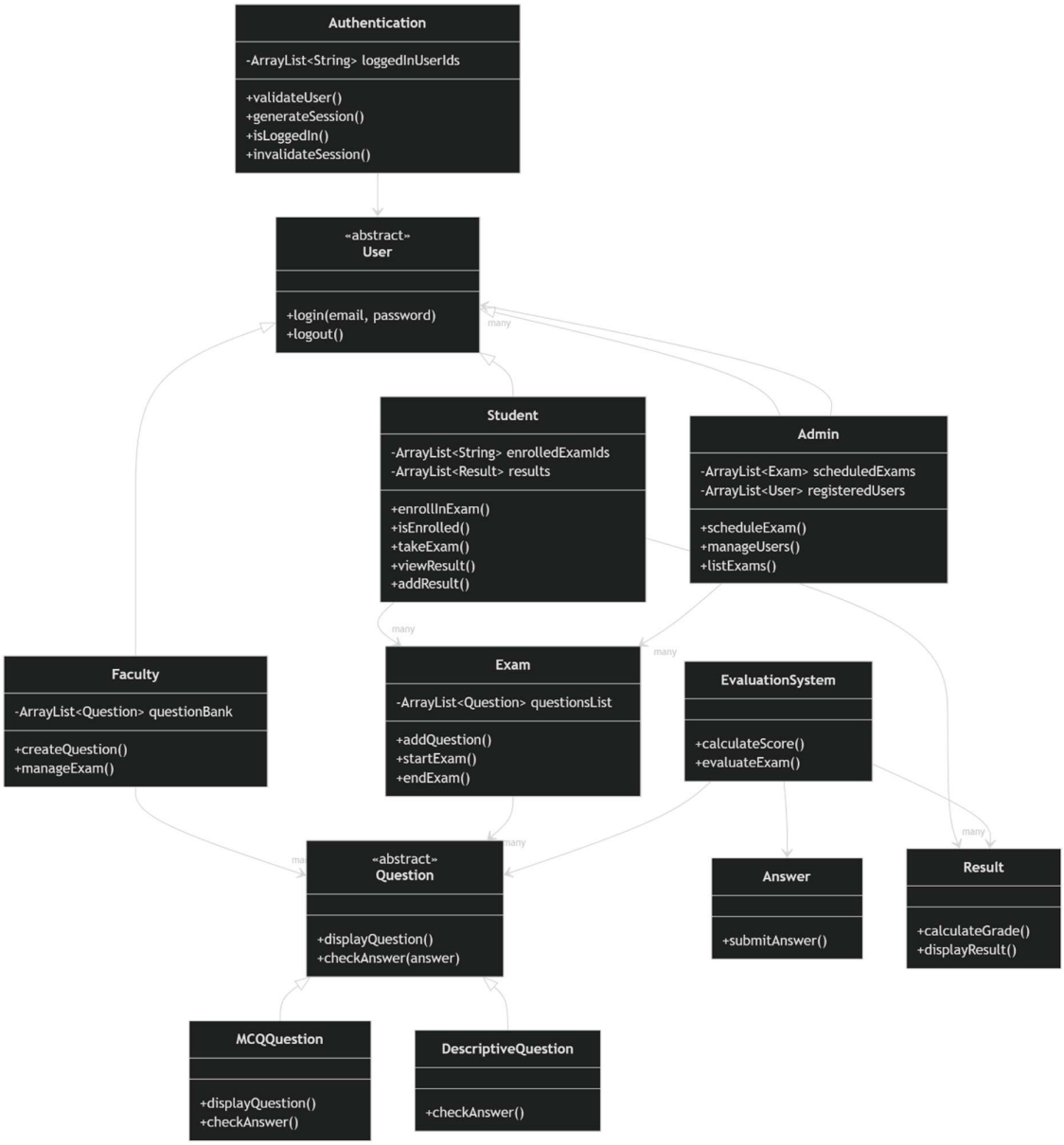
Roll No	Student Name
AM.SC.U4AIE25020	G.Mokshith
AM.SC.U4AIE25050	D.V.R.Krishna
AM.SC.U4AIE25034	Ajul TM
AM.SC.U4AIE25008	Aadith Anil

Problem Title:

Online Examination System

STRUCTURAL DESIGN

1. Class diagram (*draw class diagram that shows different classes and their relationships. Use UML notations*)



Class	Type	Key Responsibility
User	Abstract	Base identity for Student, Faculty, Admin; provides login() / logout()
Student	Concrete	Enrolls in exams, submits answers, views results; holds ArrayList<Result>

Faculty	Concrete	Creates questions into ArrayList<Question> (questionBank); loads them into an Exam
Admin	Concrete	Schedules exams (ArrayList<Exam>), registers users (ArrayList<User>)
Question	Abstract	Template for all question types; declares abstract checkAnswer() for polymorphism
MCQQuestion	Concrete	Overrides checkAnswer() — exact match against correctOption; throws on null input
DescriptiveQuestion	Concrete	Overrides checkAnswer() — keyword-count proportional scoring; throws if blank
Exam	Utility	Holds ArrayList<Question>; manages start/end lifecycle and accumulates totalMarks
Answer	Utility	Value object storing one student response; submitAnswer() prints the pair
Result	Utility	Stores score/totalMarks; calculateGrade() maps percentage to O/A+/A/B/C/F
EvaluationSystem	Utility	Iterates question-answer pairs calling polymorphic checkAnswer(); creates Result; throws if answers empty
Authentication	Utility	Tracks sessions via ArrayList<String> loggedInUserIds; generates SESSION-XXXX tokens

BEHAVIUORAL DESIGN

2. Use-case diagram
    a. List of actors
        - Student
        - Faculty
        - Admin
        - EvaluationSystem
    b. List of use-cases

Login: User enters credentials; Authentication validates using try-catch for failure.

Manage Users: Admin adds users to system using ArrayList; throws exception if null.

Schedule Exam: Admin creates and stores exam in ArrayList.

Create Question: Faculty creates questions and adds to question bank (null checked).

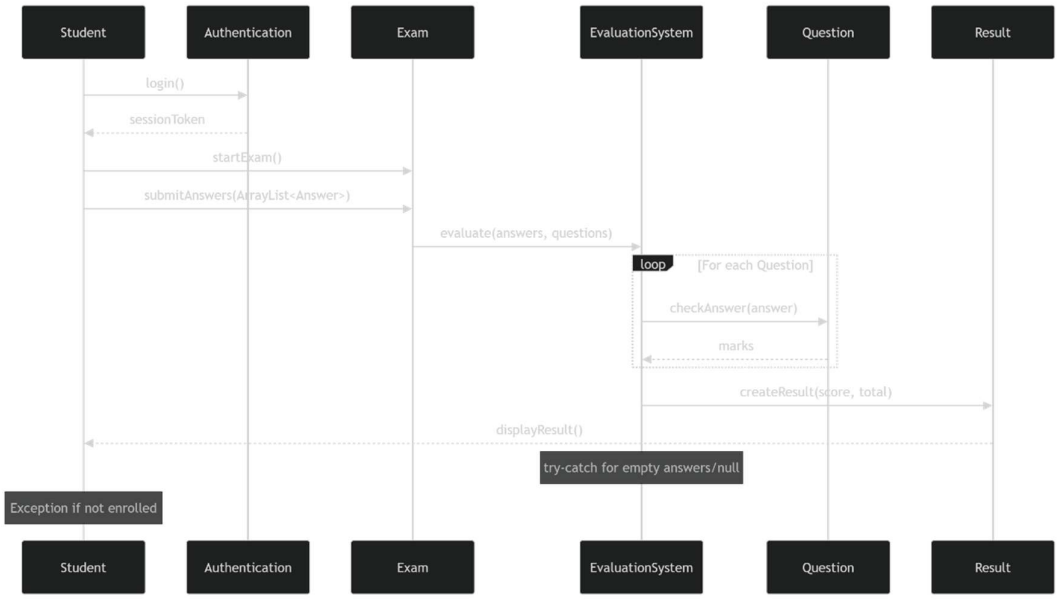
Enroll in Exam: Student enrolls by adding examId to list.

Take Exam: Student attempts exam; throws exception if not enrolled.

View Result: Student views results; exception if no results exist.

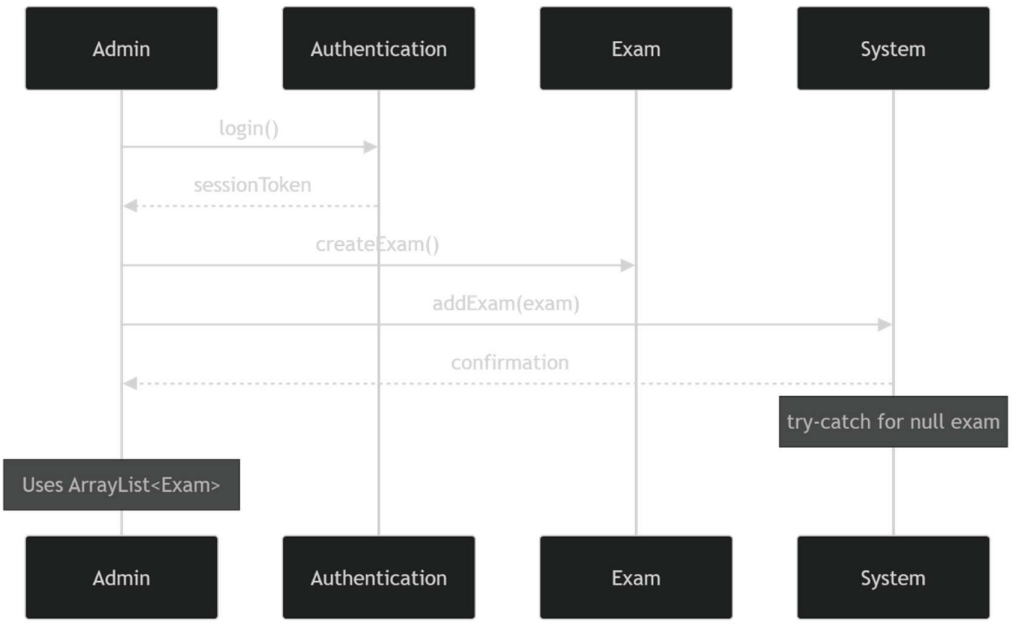
Use case diagram (use UML notations)

3. Sequence diagram (use UML notations)
   a. Choose key use cases and draw their sequence diagram. First describe the scenario in few sentences. Then draw the sequence diagram.
   Description:
   Student logs in, attempts exam, submits answers. EvaluationSystem processes answers using polymorphic checkAnswer() and generates Result. Exceptions handled if not enrolled or answers empty.



   b. Choose key use cases and draw their sequence diagram. First describe the scenario in few sentences. Then draw the sequence diagram.

Description:
Admin logs in, creates exam, and adds it to system list using ArrayList with null validation.


Admin
Authentication
Exam
System
login()
sessionToken
createExam()
addExam(exam)
confirmation
try-catch for null exam
Uses ArrayList<Exam>
Admin
Authentication
Exam
System